

Project: AI - Powered Dynamic Pricing and Demand Forecasting System

Data Collection & Preprocessing, Gather pricing data, Handle missing values, Feature engineering.

```
import os
print(os.listdir())
import pandas as pd
cab = pd.read_csv('cab_rides.csv')
weather = pd.read_csv('weather.csv')
print(cab.head())
print(weather.head())
```

```
[ '.config', 'weather.csv', 'cab_rides.csv', 'sample_data' ]
```

	distance	cab_type	time_stamp	destination	source	price	\
0	0.44	Lyft	1544952607890	North Station	Haymarket Square	5.0	
1	0.44	Lyft	1543284023677	North Station	Haymarket Square	11.0	
2	0.44	Lyft	1543366822198	North Station	Haymarket Square	7.0	
3	0.44	Lyft	1543553582749	North Station	Haymarket Square	26.0	
4	0.44	Lyft	1543463360223	North Station	Haymarket Square	9.0	

	surge_multiplier	id	product_id	\
0	1.0	424553bb-7174-41ea-aeb4-fe06d4f4b9d7	lyft_line	
1	1.0	4bd23055-6827-41c6-b23b-3c491f24e74d	lyft_premier	
2	1.0	981a3613-77af-4620-a42a-0c0866077d1e	lyft	
3	1.0	c2d88af2-d278-4bfd-a8d0-29ca77cc5512	lyft_luxsuv	
4	1.0	e0126e1f-8ca9-4f2e-82b3-50505a09db9a	lyft_plus	

	name
0	Shared
1	Lux
2	Lyft
3	Lux Black XL
4	Lyft XL

	temp	location	clouds	pressure	rain	time_stamp	humidity	\
0	42.42	Back Bay	1.0	1012.14	0.1228	1545003901	0.77	
1	42.43	Beacon Hill	1.0	1012.15	0.1846	1545003901	0.76	
2	42.50	Boston University	1.0	1012.15	0.1089	1545003901	0.76	
3	42.11	Fenway	1.0	1012.13	0.0969	1545003901	0.77	
4	43.13	Financial District	1.0	1012.14	0.1786	1545003901	0.75	

	wind
0	11.25
1	11.32
2	11.07
3	11.09
4	11.49

Demand Prediction, Predict future demand, Analyze seasonal trends.

```
# =====
# MEMBER 2 - DEMAND PREDICTION
# Using cab_rides.csv + weather.csv
# =====

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

# =====
# STEP 1: LOAD DATA
# =====

cab = pd.read_csv("cab_rides.csv")
weather = pd.read_csv("weather.csv")

print("Cab Shape:", cab.shape)
print("Weather Shape:", weather.shape)

# =====
# STEP 2: CONVERT TIMESTAMPS
# =====

cab["datetime"] = pd.to_datetime(
    cab["time_stamp"],
    unit="ms"
)
```

```

weather["datetime"] = pd.to_datetime(
    weather["time_stamp"],
    unit="s"
)

# =====
# STEP 3: CREATE HOUR COLUMN
# =====

cab["hour"] = cab["datetime"].dt.hour
cab["date"] = cab["datetime"].dt.date

weather["hour"] = weather["datetime"].dt.hour
weather["date"] = weather["datetime"].dt.date

# =====
# STEP 4: CREATE DEMAND DATASET
# Demand = Number of rides per hour
# =====

demand = (
    cab.groupby(["date", "hour"])
    .size()
    .reset_index(name="ride_count")
)

print("\nDemand Dataset")
print(demand.head())

# =====
# STEP 5: AGGREGATE WEATHER DATA
# =====

weather_hourly = (
    weather.groupby(["date", "hour"])
    .agg({
        "temp": "mean",
        "humidity": "mean",
        "pressure": "mean",
        "wind": "mean",
        "clouds": "mean",
        "rain": "mean"
    })
    .reset_index()
)

# =====
# STEP 6: MERGE DEMAND + WEATHER
# =====

data = pd.merge(
    demand,
    weather_hourly,
    on=["date", "hour"],
    how="left"
)

# Fill missing values
data.fillna(data.median(numeric_only=True), inplace=True)

print("\nMerged Dataset")
print(data.head())

# =====
# STEP 7: DEMAND TREND ANALYSIS
# =====

hourly = data.groupby("hour")["ride_count"].mean()

plt.figure(figsize=(8,5))
hourly.plot()
plt.title("Average Demand by Hour")
plt.xlabel("Hour")
plt.ylabel("Ride Count")
plt.grid()
plt.show()

# =====
# STEP 8: CREATE LAG FEATURES
# =====

data = data.sort_values(["date", "hour"])

```

```

data["lag_1"] = data["ride_count"].shift(1)
data["lag_2"] = data["ride_count"].shift(2)
data["lag_3"] = data["ride_count"].shift(3)

data.dropna(inplace=True)

# =====
# STEP 9: FEATURES & TARGET
# =====

X = data[
    [
        "hour",
        "temp",
        "humidity",
        "pressure",
        "wind",
        "clouds",
        "rain",
        "lag_1",
        "lag_2",
        "lag_3"
    ]
]

y = data["ride_count"]

# =====
# STEP 10: TRAIN TEST SPLIT
# =====

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)

# =====
# STEP 11: TRAIN MODEL
# =====

model = RandomForestRegressor(
    n_estimators=100,
    random_state=42
)

model.fit(X_train, y_train)

# =====
# STEP 12: PREDICT
# =====

pred = model.predict(X_test)

# =====
# STEP 13: EVALUATION
# =====

mae = mean_absolute_error(y_test, pred)

rmse = np.sqrt(
    mean_squared_error(y_test, pred)
)

r2 = r2_score(y_test, pred)

print("\n==== MODEL RESULTS =====")
print("MAE :", round(mae,2))
print("RMSE:", round(rmse,2))
print("R2  :", round(r2,2))

# =====
# STEP 14: ACTUAL VS PREDICTED
# =====

plt.figure(figsize=(10,5))

plt.plot(
    y_test.values,
    label="Actual Demand"

```

```
)

plt.plot(
    pred,
    label="Predicted Demand"
)

plt.title("Actual vs Predicted Demand")
plt.xlabel("Samples")
plt.ylabel("Ride Count")
plt.legend()
plt.show()

# =====
# STEP 15: FEATURE IMPORTANCE
# =====

importance = pd.DataFrame({
    "Feature": X.columns,
    "Importance": model.feature_importances_
})

importance = importance.sort_values(
    by="Importance",
    ascending=False
)

print("\nFeature Importance")
print(importance)

# =====
# STEP 16: FUTURE DEMAND PREDICTION
# =====

latest = data.iloc[-1]

future_input = pd.DataFrame({
    "hour": [18],
    "temp": [latest["temp"]],
    "humidity": [latest["humidity"]],
    "pressure": [latest["pressure"]],
    "wind": [latest["wind"]],
    "clouds": [latest["clouds"]],
    "rain": [latest["rain"]],
    "lag_1": [latest["ride_count"]],
    "lag_2": [latest["lag_1"]],
    "lag_3": [latest["lag_2"]]
})

future_demand = model.predict(future_input)

print("\nPredicted Future Demand:")
print(round(future_demand[0]))
```



```
Cab Shape: (693071, 10)
Weather Shape: (6276, 8)
```

```
Demand Dataset
   date  hour  ride_count
0  2018-11-26    3         84
1  2018-11-26    4        420
2  2018-11-26    5        672
3  2018-11-26    6       1596
4  2018-11-26    7       1008
```

Dynamic Price Prediction, Regression models, Price optimization.

```

# =====
# MEMBER 3 - DYNAMIC PRICE PREDICTION
# =====

import pandas as pd
import numpy as np

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.ensemble import RandomForestRegressor
from sklearn.linear_model import LinearRegression
from sklearn.neural_network import MLPRegressor
from sklearn.preprocessing import StandardScaler

from sklearn.metrics import (
    mean_absolute_error,
    mean_squared_error,
    r2_score
)

# =====
# STEP 1: LOAD DATA
# =====

cab = pd.read_csv("cab_rides.csv")

print("Original Shape:", cab.shape)

# =====
# STEP 2: REMOVE MISSING PRICES
# =====

cab = cab.dropna(subset=["price"])

print("After Removing Null Prices:", cab.shape)

# =====
# STEP 3: CREATE DATE & HOUR
# =====

cab["datetime"] = pd.to_datetime(
    cab["time_stamp"],
    unit="ms"
)

cab["hour"] = cab["datetime"].dt.hour
cab["date"] = cab["datetime"].dt.date

# =====
# STEP 4: CREATE DEMAND FEATURE
# Demand = rides per hour
# =====

demand = (
    cab.groupby(["date", "hour"])
    .size()
    .reset_index(name="ride_count")
)

# Merge demand back

cab = pd.merge(
    cab,
    demand,
    on=["date", "hour"],
    how="left"
)

print("After Demand Merge:", cab.shape)
```

```
# =====
# STEP 5: KEEP USEFUL COLUMNS
# =====

cab = cab[
    [
        "distance",
        "surge_multiplier",
        "ride_count",
        "cab_type",
        "source",
        "destination",
        "name",
        "price"
    ]
]

# =====
# STEP 6: ENCODE CATEGORICAL DATA
# =====

cols = [
    "cab_type",
    "source",
    "destination",
    "name"
]

encoders = {}

for col in cols:

    le = LabelEncoder()

    cab[col] = le.fit_transform(cab[col])

    encoders[col] = le

# =====
# STEP 7: FEATURES & TARGET
# =====

X = cab.drop("price", axis=1)

y = cab["price"]

# =====
# STEP 8: TRAIN TEST SPLIT
# =====

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)

# =====
# STEP 9: LINEAR REGRESSION
# =====

lr = LinearRegression()

lr.fit(X_train, y_train)

lr_pred = lr.predict(X_test)

print("\n==== LINEAR REGRESSION =====")

print(
    "MAE:",
    round(mean_absolute_error(y_test, lr_pred), 2)
)

print(
    "RMSE:",
    round(np.sqrt(mean_squared_error(y_test, lr_pred)), 2)
)

print(
    "R2:",
```

```
        round(r2_score(y_test, lr_pred),2)
    )

# =====
# STEP 10: RANDOM FOREST
# =====

rf = RandomForestRegressor(
    n_estimators=100,
    random_state=42,
    n_jobs=-1
)

rf.fit(X_train, y_train)

rf_pred = rf.predict(X_test)

print("\n===== RANDOM FOREST =====")

print(
    "MAE:",
    round(mean_absolute_error(y_test, rf_pred),2)
)

print(
    "RMSE:",
    round(np.sqrt(mean_squared_error(y_test, rf_pred)),2)
)

print(
    "R2:",
    round(r2_score(y_test, rf_pred),2)
)

# =====
# STEP 10B: NEURAL NETWORK
# =====

scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

nn = MLPRegressor(
    hidden_layer_sizes=(128,64),
    activation="relu",
    solver="adam",
    max_iter=300,
    random_state=42
)

nn.fit(
    X_train_scaled,
    y_train
)

nn_pred = nn.predict(
    X_test_scaled
)

print("\n===== NEURAL NETWORK =====")

print(
    "MAE:",
    round(mean_absolute_error(y_test, nn_pred),2)
)

print(
    "RMSE:",
    round(
        np.sqrt(
            mean_squared_error(
                y_test,
                nn_pred
            )
        ),
        2
    )
)

print(
    "R2:",
```



```

        round(
            r2_score(
                y_test,
                nn_pred
            ),
            2
        )
    )
)

print("\n==== MODEL COMPARISON =====")

comparison = pd.DataFrame({

    "Model":[
        "Linear Regression",
        "Random Forest",
        "Neural Network"
    ],

    "R2":[
        round(r2_score(y_test, lr_pred),4),
        round(r2_score(y_test, rf_pred),4),
        round(r2_score(y_test, nn_pred),4)
    ]
})

print(comparison)


# =====
# STEP 11: FEATURE IMPORTANCE
# =====

importance = pd.DataFrame({
    "Feature": X.columns,
    "Importance": rf.feature_importances_
})

importance = importance.sort_values(
    by="Importance",
    ascending=False
)

print("\nFeature Importance")
print(importance)


# =====
# STEP 12: USE MEMBER 2 OUTPUT
# =====

future_demand = 1413

sample_input = pd.DataFrame({

    "distance":[5.0],

    "surge_multiplier":[1.2],

    "ride_count":[future_demand],

    "cab_type":[
        encoders["cab_type"].transform(["Lyft"])[0]
    ],

    "source":[
        encoders["source"].transform(
            ["Haymarket Square"]
        )[0]
    ],

    "destination":[
        encoders["destination"].transform(
            ["North Station"]
        )[0]
    ],

    "name":[
        encoders["name"].transform(
            ["Lyft"]
        )[0]
    ]
})

```

```

sample_scaled = scaler.transform(
    sample_input
)

recommended_price = nn.predict(
    sample_scaled
)

print("\n=====")
print("Predicted Future Demand:", future_demand)
print(
    "Recommended Price: $",
    round(recommended_price[0],2)
)
print("=====")

```

```

Original Shape: (693071, 10)
After Removing Null Prices: (637976, 10)
After Demand Merge: (637976, 14)

```

```

===== LINEAR REGRESSION =====
MAE: 5.16
RMSE: 6.49
R2: 0.52

```

```

===== RANDOM FOREST =====
MAE: 1.13
RMSE: 1.8
R2: 0.96

```

```

/usr/local/lib/python3.12/dist-packages/sklearn/neural_network/_multilayer_perceptron.py:698: UserWarning: Training interrupted by user.
warnings.warn("Training interrupted by user.")

```

```

===== NEURAL NETWORK =====
MAE: 1.14
RMSE: 1.71
R2: 0.97

```

```

===== MODEL COMPARISON =====
              Model      R2
0  Linear Regression  0.5169
1      Random Forest  0.9629
2      Neural Network  0.9664

```

```

Feature Importance
              Feature  Importance
6              name    0.773020
0            distance  0.152739
1  surge_multiplier  0.040819
3            cab_type  0.013598
2            ride_count  0.013341
5            destination  0.003436
4              source    0.003049

```

```

=====
Predicted Future Demand: 1413
Recommended Price: $ 17.73
=====

```

Competitor Analysis, Compare competitor prices, Market trend analysis.

```

# =====
# MEMBER 4 - COMPETITOR ANALYSIS
# MARKET TREND ANALYSIS
# =====

import pandas as pd
import matplotlib.pyplot as plt

# =====
# STEP 1: LOAD DATA
# =====

cab = pd.read_csv("cab_rides.csv")

print("Original Shape:", cab.shape)

# =====
# STEP 2: REMOVE MISSING PRICES
# =====

cab = cab.dropna(subset=["price"])

print("After Cleaning:", cab.shape)

```

```
# =====
# STEP 3: CREATE DATETIME FEATURES
# =====

cab["datetime"] = pd.to_datetime(
    cab["time_stamp"],
    unit="ms"
)

cab["hour"] = cab["datetime"].dt.hour
cab["day"] = cab["datetime"].dt.day_name()

# =====
# STEP 4: COMPETITOR PRICE ANALYSIS
# =====

print("\n==== AVERAGE PRICE BY CAB TYPE =====")

cab_type_price = (
    cab.groupby("cab_type")["price"]
    .mean()
    .sort_values(ascending=False)
)

print(cab_type_price)

# =====
# STEP 5: VISUALIZE CAB TYPE PRICES
# =====

plt.figure(figsize=(10,5))

cab_type_price.plot(kind="bar")

plt.title("Average Price by Cab Type")
plt.ylabel("Average Price")
plt.xlabel("Cab Type")

plt.show()

# =====
# STEP 6: SURGE MULTIPLIER ANALYSIS
# =====

print("\n==== SURGE ANALYSIS =====")

surge_analysis = (
    cab.groupby("surge_multiplier")["price"]
    .mean()
)

print(surge_analysis)

plt.figure(figsize=(8,5))

surge_analysis.plot(marker="o")

plt.title("Price vs Surge Multiplier")
plt.xlabel("Surge Multiplier")
plt.ylabel("Average Price")

plt.grid()

plt.show()

# =====
# STEP 7: PEAK HOUR PRICE ANALYSIS
# =====

print("\n==== PEAK HOUR ANALYSIS =====")

hourly_price = (
    cab.groupby("hour")["price"]
    .mean()
)

print(hourly_price)

plt.figure(figsize=(10,5))

hourly_price.plot()
```

```

plt.title("Average Price by Hour")
plt.xlabel("Hour")
plt.ylabel("Average Price")

plt.grid()

plt.show()

# =====
# STEP 8: MOST EXPENSIVE ROUTES
# =====

print("\n==== TOP 10 EXPENSIVE ROUTES =====")

routes = (
    cab.groupby(
        ["source", "destination"]
    )["price"]
    .mean()
    .reset_index()
)

routes = routes.sort_values(
    by="price",
    ascending=False
)

print(routes.head(10))

# =====
# STEP 9: DISTANCE VS PRICE
# =====

plt.figure(figsize=(8,5))

plt.scatter(
    cab["distance"],
    cab["price"],
    alpha=0.2
)

plt.title("Distance vs Price")
plt.xlabel("Distance")
plt.ylabel("Price")

plt.show()

# =====
# STEP 10: MARKET INSIGHTS
# =====

highest_cab = cab_type_price.idxmax()
highest_price = cab_type_price.max()

lowest_cab = cab_type_price.idxmin()
lowest_price = cab_type_price.min()

print("\n==== MARKET INSIGHTS =====")

print(
    f"Most Expensive Service: "
    f"{highest_cab} "
    f"${highest_price:.2f}"
)

print(
    f"Least Expensive Service: "
    f"{lowest_cab} "
    f"${lowest_price:.2f}"
)

peak_hour = hourly_price.idxmax()

print(
    f"Peak Pricing Hour: "
    f"{peak_hour}:00"
)

best_route = routes.iloc[0]

print(
    f"Most Expensive Route: "
    f"{best_route['source']} → "

```

```
        f"{best_route['destination']}")
    )

# =====
# STEP 11: COMPETITIVE STRATEGY
# =====

our_price = 16.8

market_avg = cab["price"].mean()

print("\n==== STRATEGY REPORT =====")

print(
    f"Market Average Price: "
    f"${market_avg:.2f}"
)

print(
    f"Our Predicted Price: "
    f"${our_price:.2f}"
)

if our_price < market_avg:

    print(
        "Strategy: Competitive Pricing"
    )

elif our_price > market_avg:

    print(
        "Strategy: Premium Pricing"
    )

else:

    print(
        "Strategy: Market Matching"
    )
```



```
Original Shape: (693071, 10)
After Cleaning: (637976, 10)
```

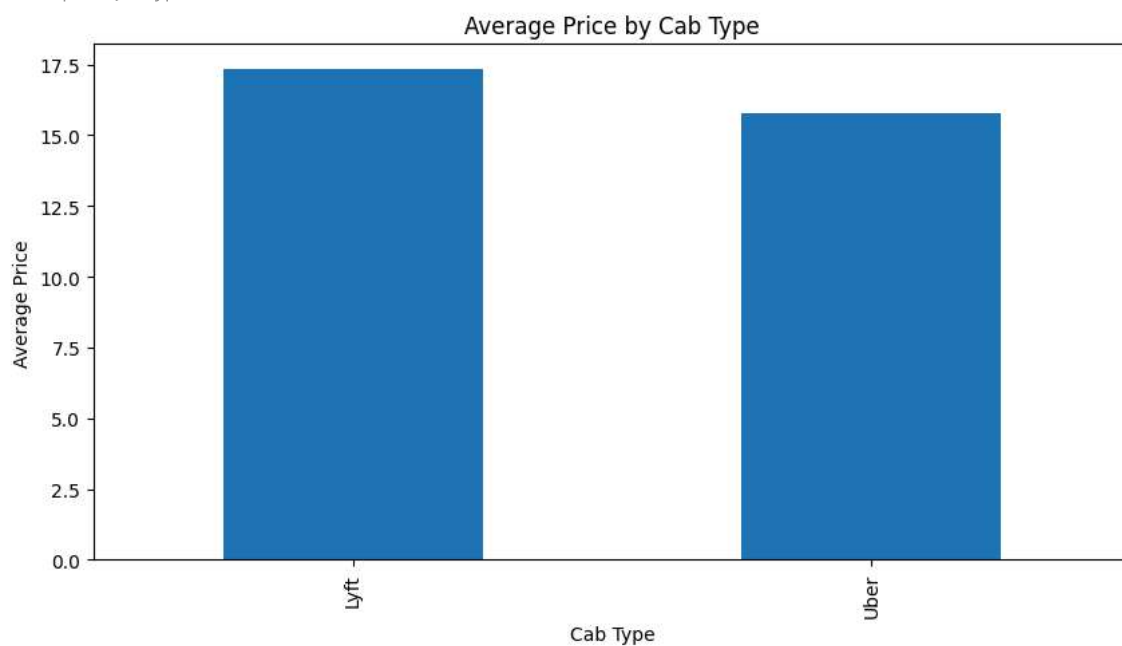
```
===== AVERAGE PRICE BY CAB TYPE =====
```

```
cab_type
```

```
Lyft    17.351396
```

```
Uber    15.795343
```

```
Name: price, dtype: float64
```



```
===== SURGE ANALYSIS =====
```

```
surge_multiplier
```

```
1.00    16.161259
```

```
1.25    23.768471
```

```
1.50    28.894472
```

```
1.75    33.673760
```

```
2.00    39.048459
```

```
2.50    30.207792
```

```
3.00    40.333333
```

```
Name: price, dtype: float64
```



```
===== PEAK HOUR ANALYSIS =====
```

```
hour
```

```
0    16.574769
```

```
1    16.513778
```

```
2    16.561803
```

```
3    16.539718
```

```
4    16.579311
```

```
5    16.537217
```

```
6    16.519326
```

```
7    16.491627
```

```
8    16.603326
```

```
9    16.512578
```

```
10   16.501021
```

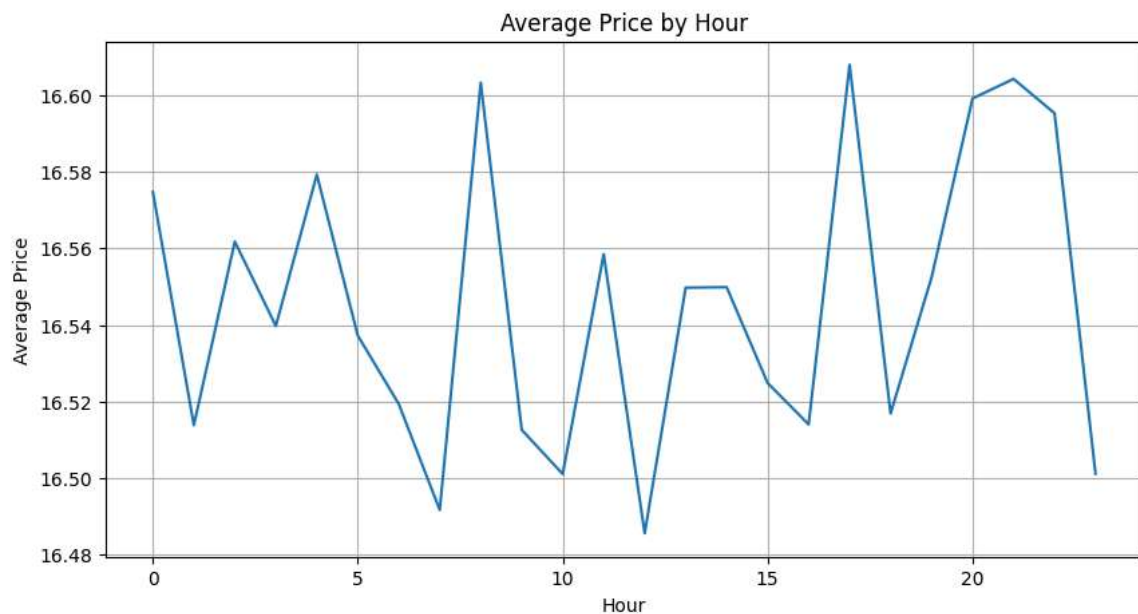
```
11   16.558506
```

```
12   16.485527
```

```

13 16.549744
14 16.549882
15 16.524813
16 16.513991
17 16.607996
18 16.516847
19 16.552437
20 16.599208
21 16.604323
22 16.595357
23 16.501090
Name: price, dtype: float64

```



===== TOP 10 EXPENSIVE ROUTES =====

	source	destination	price
24	Financial District	Boston University	25.498434
14	Boston University	Financial District	24.146085
20	Fenway	Financial District	23.438818
25	Financial District	Fenway	23.404850
50	Northeastern University	Financial District	22.582094
28	Financial District	Northeastern University	21.918584
60	Theatre District	Boston University	20.360662
15	Boston University	North Station	20.185338
51	Northeastern University	North Station	19.910939
21	Fenway	North Station	19.701839



===== MARKET INSIGHTS =====

Most Expensive Service: Lyft (\$17.35)
 Least Expensive Service: Uber (\$15.80)
 Peak Pricing Hour: 17:00
 Most Expensive Route: Financial District → Boston University

===== STRATEGY REPORT =====

Market Average Price: \$16.55
 Our Predicted Price: \$16.80
 Strategy: Premium Pricing

Revenue Optimization, Calculate expected revenue, Recommend best pricing strategy.

```
# =====
# MEMBER 5 - REVENUE OPTIMIZATION
# =====

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# =====
# INPUTS FROM MEMBER 2 & MEMBER 3
# =====

predicted_demand = 1413

recommended_price = 16.80

# =====
# INPUT SUMMARY
# =====

print("="*50)
print("INPUT SUMMARY")
print("="*50)

print(f"Predicted Demand : {predicted_demand}")
print(f"Recommended Price: ${recommended_price}")

# =====
# BASE REVENUE
# =====

base_revenue = (
    predicted_demand
    * recommended_price
)

print("\nBase Revenue: $",
      round(base_revenue,2))

# =====
# PRICE SCENARIOS
# =====

price_scenarios = np.arange(
    recommended_price - 3,
    recommended_price + 3.5,
    0.5
)

results = []

# =====
# DEMAND ELASTICITY MODEL
#
# Every $1 increase:
# demand drops by 5%
#
# Every $1 decrease:
# demand increases by 5%
# =====

for price in price_scenarios:

    price_change = (
        price
        - recommended_price
    )

    adjusted_demand = (
        predicted_demand
        * (
            1
            - (0.05 * price_change)
        )
    )

    adjusted_demand = max(
        adjusted_demand,
        0
    )
```